

가천대학교 스마트시티학과 – 창업프로젝트

Lecture 02

Urban Problem Discovery and Web Service Fundamentals

Chris Ohk

utilForever@gmail.com

학습 목표

- 문제와 해결책을 구분하고 사용자 / 상황 / 불만을 설명한다.
- 사용자와 운영자, 데이터 제공자 등 이해 관계자를 구분한다.
- URL과 HTTP 요청 / 응답이 페이지를 만드는 과정을 설명한다.
- 각자 HTML을 작성하고 브라우저에서 검색 요청을 확인한다.

기술보다 문제 먼저

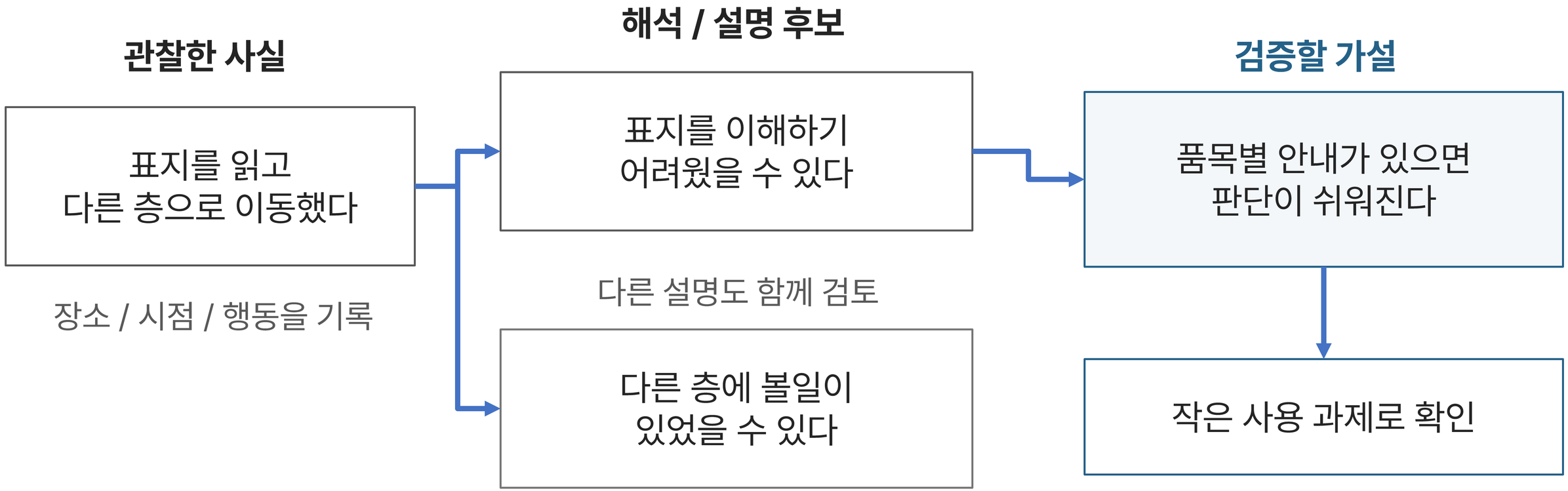
- 출발점 : 누가, 언제, 무엇 때문에 불편한 게 생기는가?
- 문제 : 사용자가 겪는 어려움과 그 결과
- 해결책 : 그 어려움을 줄이려는 방법
- 만들 수 있다는 사실만으로 필요성이 입증되지는 않는다.

가상의 캠퍼스 사례

- 상황 : 수업 직전, 학생이 빈 음료 용기를 들고 있다.
- 장면 : A 도서관 로비에서 표지를 살핀 뒤 다른 층으로 이동한다.
- 아직 모름 : 이동 이유, 불편 정도, 반복 여부

관찰 / 해석 / 가설을 구분하기

가상 기록에서 출발해, 설명 후보를 나누고 검증한다.



관찰은 기록, 해석은 설명 후보, 가설은 아직 확인하지 않은 주장이다.

행동에서 막히는 순간 찾기

- 행동 흐름 : 용기를 확인하고 안내를 찾아 이동한 뒤 배출한다.
- 막힘 : 품목 구분, 위치 확인, 이용 가능 여부
- 영향 : 추가 이동, 시간 지연, 판단 포기 가능성
- 확인할 것 : 어느 단계에서, 누구에게 반복되는가?

우회 행동은 문제의 단서

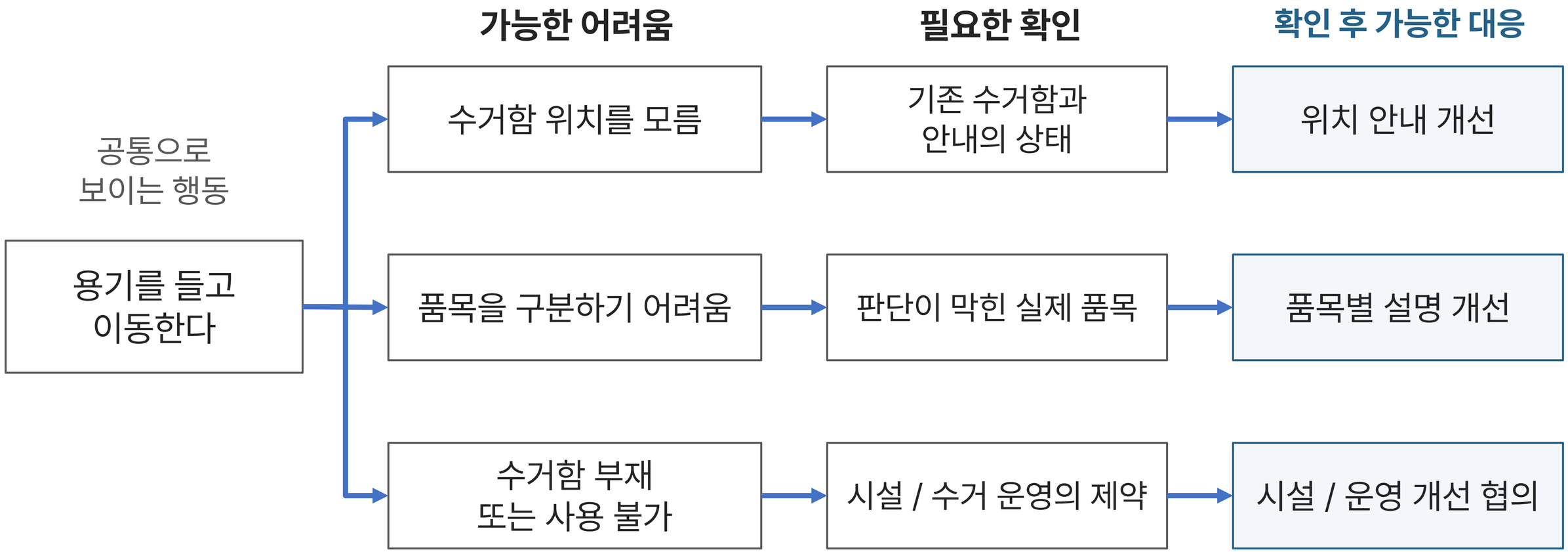
- 우회 행동 : 친구에게 묻기, 다른 층 가기, 사진 저장
- 질문 : 지금은 어떤 방법으로 해결하고 있는가?
- 비용 : 시간 / 수고 / 불확실성 중 무엇을 감수하는가?
- 기존 방법이 충분하다면 새 서비스의 이유도 약해진다

문제를 한 문장으로 쓰기

- 사용자 + 상황 + 어려움 + 영향
- 예시 : 낯선 건물에서 수업을 기다리는 학생은 음료 용기를 버릴 위치를 확인하기 어려워 추가 이동을 하거나 처리를 미룬다.
- 가설 문장 : 각 요소를 실제 관찰 / 질문으로 확인해야 한다.

같은 현상, 다른 문제

같은 행동이라도 원인은 다를 수 있다.



세 갈래는 경쟁하는 설명이다. 실제 원인을 확인한 결과가 아니다.

인터뷰는 최근 행동부터

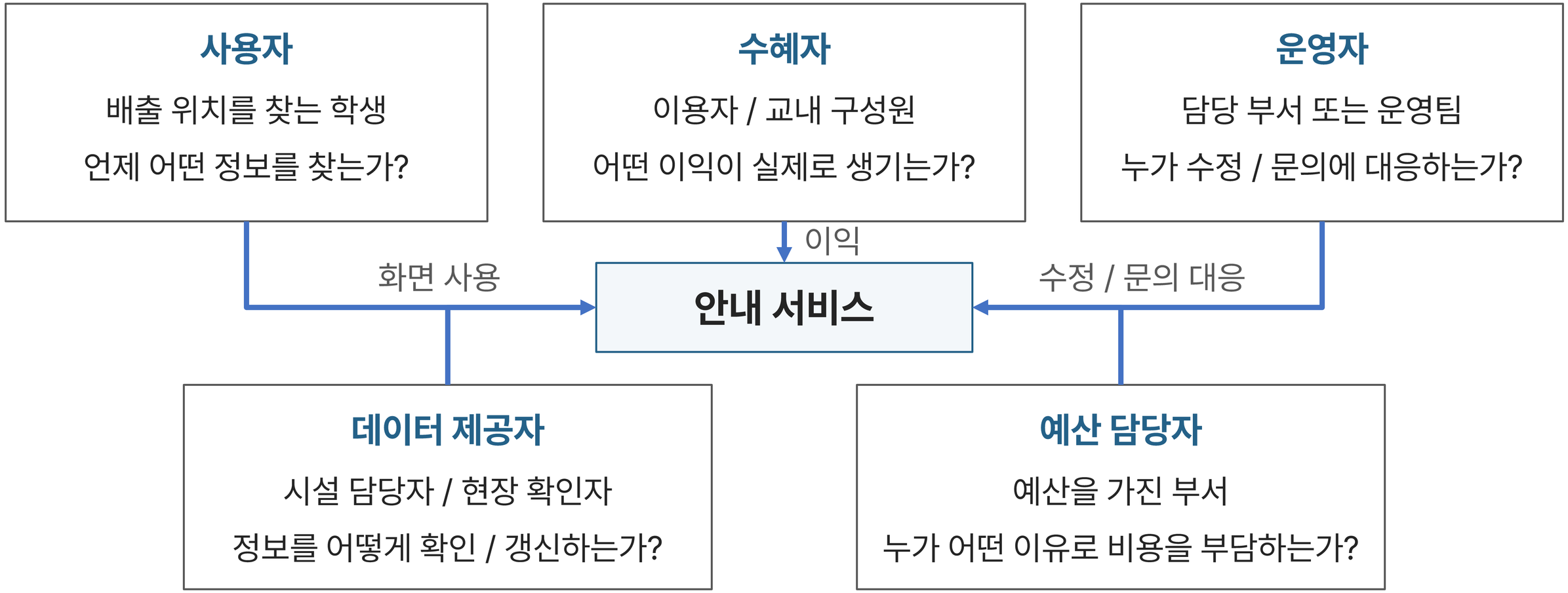
- “최근 캠퍼스에서 음료 용기를 버린 때를 말해 주세요.”
- “어디에 있었고, 먼저 무엇을 했나요?”
- “무엇을 보고 판단했나요? 그다음에는요?”
- “어려움이 없었던 경우도 있었나요?”
- 행동 / 인용 / 해석을 나누어 기록하고, 기록 동의를 구한다

답을 유도하지 않는 질문

유도하기 쉬운 질문	행동을 묻는 질문
안내가 불편해서 헤맸죠?	그때 어떤 순서로 이동했나요?
우리 웹이 있으면 쓰겠죠?	최근 위치를 확인할 때 무엇을 썼나요?
기능을 더 넣으면 좋겠죠?	현재 방법에서 해결되지 않는 일이 있나요?

누가 쓰고, 누가 운영하는가

역할별 후보와 질문을 함께 확인한다



가치와 비용을 함께 보기

- 사용자 가치 : 필요한 정보를 찾는 수고를 줄이는가?
- 운영 부담 : 확인 / 갱신 / 문의에 누가 시간을 쓰는가?
- 지불 이유 : 담당 조직의 목표와 예산에 맞는가?
- 지속 가능성: 얻는 가치와 유지 비용을 설명할 수 있는가?

무엇을 확인할지 먼저 정하기

- 문제 가설 : 위치 확인이 실제로 막히는 순간이 있는가?
- 가치 가설 : 제공한 정보가 다음 행동에 도움이 되는가?
- 사용성 가설 : 설명 없이 필요한 안내를 찾을 수 있는가?
- 운영 가설 : 정보를 확인하고 갱신할 담당자가 있는가?
- 가설의 종류에 따라 필요한 근거와 확인 방법이 달라진다

작게 시작할 과제 고르기

후보	근거 확인	정보 접근	지금의 구현 범위
품목별 위치 안내	위치 탐색 어려움 확인 필요	현장 / 담당자 확인 필요	정적 목록부터 가능
자동 품목 판별	어려운 품목부터 확인 필요	정확한 기준 / 검증 자료 필요	오류 대응까지 고려
실시간 수거함 상태	상태 정보 필요성 확인	센서 / 현장 운영 협조 필요	현재 실습 범위 초과

웹이 바꿀 수 있는 범위

- 할 수 있는 부분 : 등록된 품목에 맞는 배출 위치를 전달
- 확인이 필요한 부분 : 안내가 실제 판단과 이동에 도움이 되는지
- 웹만으로 어려운 부분 : 수거함 설치 / 수거 / 청소 / 시설 개방
- 안내 개선과 시설 / 운영 개선은 각각 책임 주체가 필요

첫 사용 장면을 구체화하기

- 사용자 : 처음 방문한 건물에서 배출 위치를 찾는 학생
- 상황 : 다음 일정 전, 휴대전화로 안내를 확인
- 행동 : 배출할 품목을 선택하고 안내된 위치를 살핀다
- 약속 : 선택한 품목에 등록된 배출 위치를 읽기 쉽게 전달

최소 범위와 제외 범위

- 최소 범위 : 품목 선택, 위치 목록, 전체 보기, 결과 없음 표시
- 가상 데이터 : 종이 2곳, 플라스틱 2곳, 캔 1곳, 유리 0곳
- 필요 정보 : 각 위치의 건물 / 층 / 장소와 취급 품목
- 후속 검증 : 이용 조건 / 확인일 / 갱신 책임 / 오류 신고 운영
- 제외 범위 : 로그인 / 포인트 / 자동 판별 / 실시간 센서

가설을 요구사항으로 바꾸기

사용자에게 필요한 것	이번 화면의 요구사항	완료 확인: 시작 데이터
품목에 맞는 위치 찾기	품목을 선택하면 해당 위치 목록 표시	종이 2곳, 플라스틱 2곳, 캔 1곳
등록된 장소를 읽고 비교	건물 / 층 / 장소 표시, 전체 보기 제공	전체 5곳이며 장소명이 자료와 일치
안내의 한계 이해	가상 자료 / 결과 없음 상태 표시	유리 0곳을 시설 부재로 단정하지 않음

예제 : 문제에서 구현 범위로

- 문제 : 낯선 건물에서 배출 위치를 확인하기 어렵다.
- 입력 : 버릴 품목을 선택한다.
- 출력 : 등록된 건물 / 층 / 장소를 표시한다.
- 범위 : 품목별 위치 조회와 결과 없음 안내를 구현한다.
- 한계 : 거리 / 출입 조건을 알 수 없다.

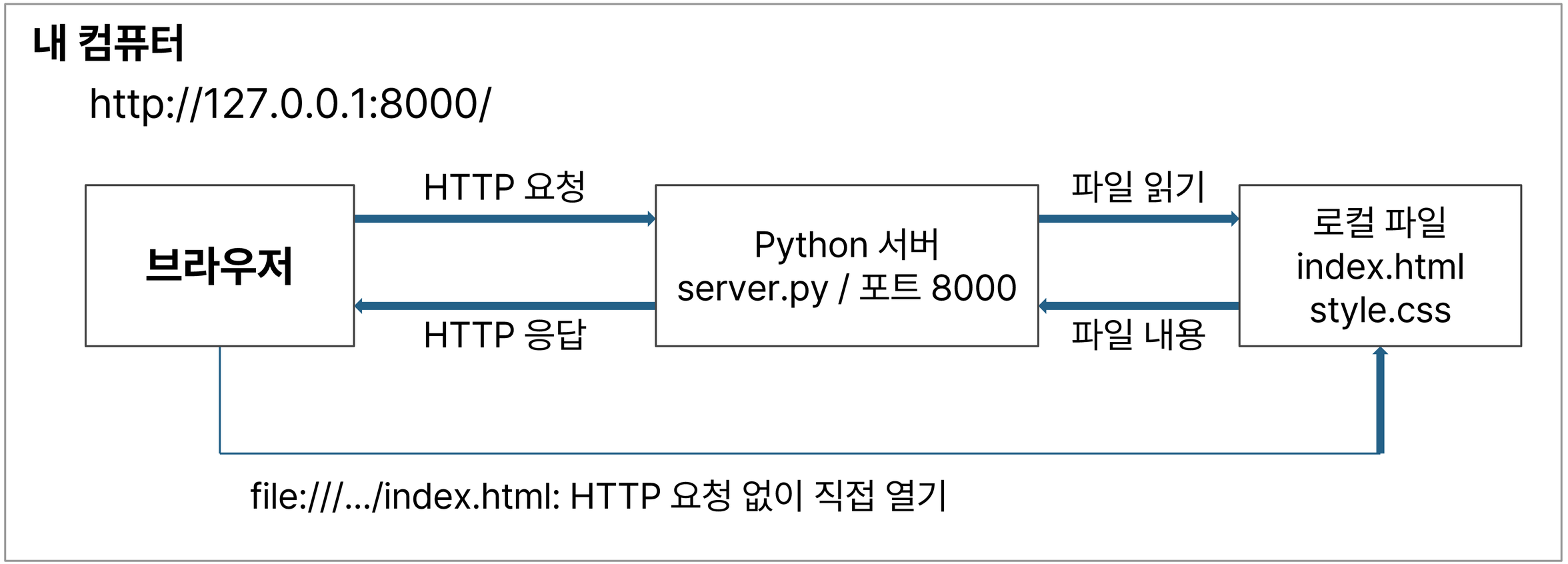
개인 프로젝트의 구현 범위

- 폴더에서 수정 / 실행 / 확인한다.
- index.html : 제목 / 문단 / 목록 / 표 / 링크 / 검색 폼 작성
- data.py : 제공된 Python 리스트에 가상 장소 한 건 추가
- server.py와 style.css : 제공된 서버와 스타일 사용
- solution : 완성된 HTML과 동작을 비교할 예시

브라우저, 클라이언트, 서버

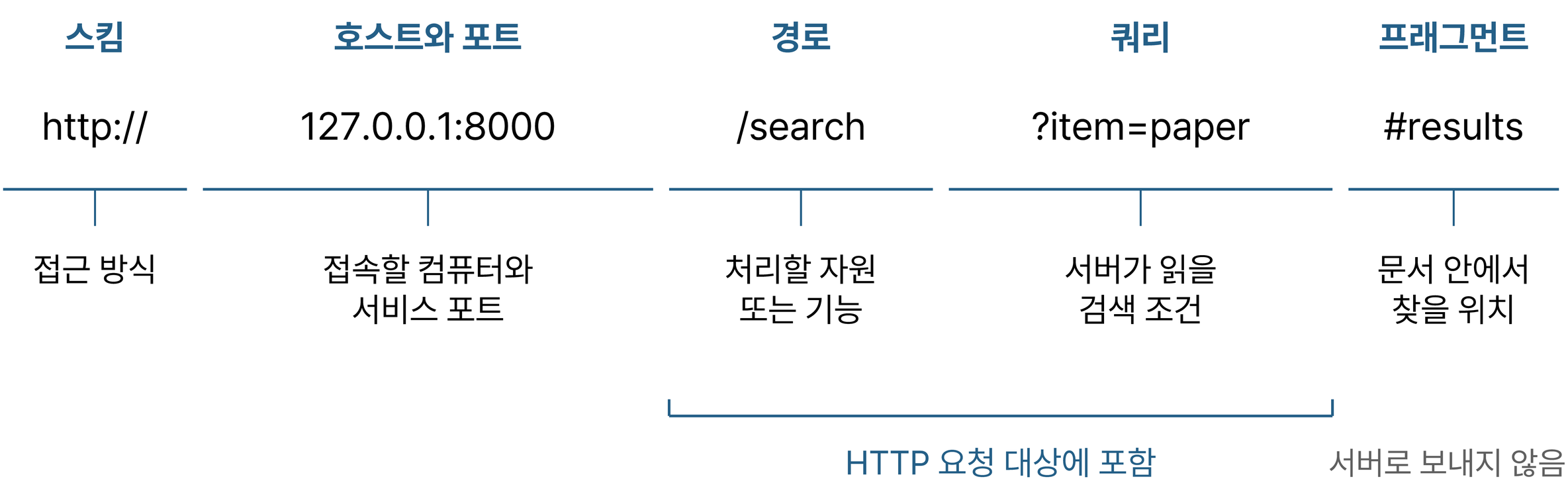
- 사용자는 브라우저에서 페이지를 열거나 검색 조건을 선택한다.
  - 브라우저는 HTTP 요청을 보내고 받은 HTML을 해석해 화면을 표시한다.
- 클라이언트는 서비스를 요청하는 역할, 서버는 요청을 처리하는 역할
  - 서버라는 말은 처리 프로그램이나 그 프로그램이 실행되는 컴퓨터를 가리킨다.
- 이번 실습에서는 브라우저와 Python 서버를 같은 컴퓨터에서 실행한다.

내 컴퓨터에서 실행하는 서버



URL의 구성 요소

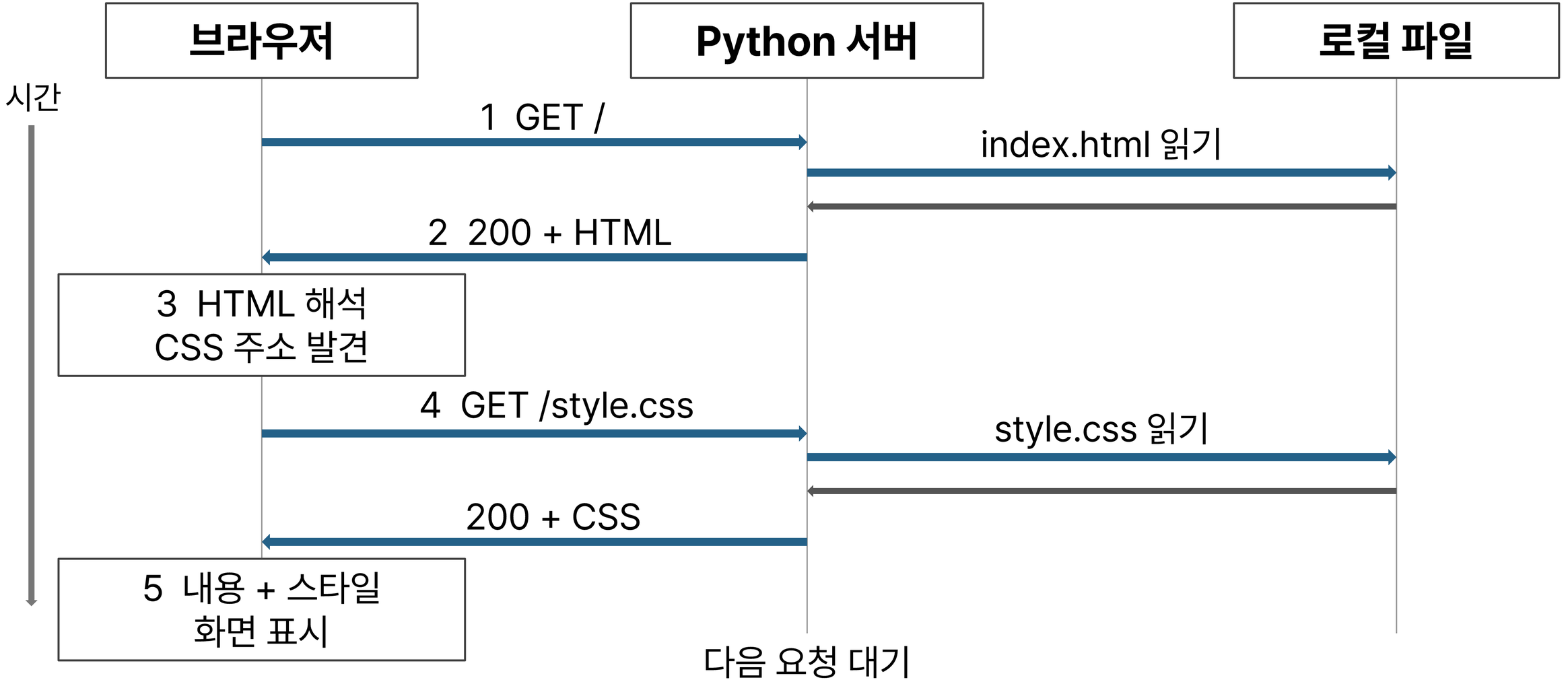
주소의 각 부분이 맡는 역할



쿼리와 프래그먼트

- `item=paper`는 서버가 종이류 안내를 선택하는 데 사용하는 조건이다.
  - `?` 뒤의 이름과 값은 요청 대상에 포함된다. 여러 조건은 보통 `&`로 구분한다.
- `#results`는 브라우저가 `id="results"`인 문서 위치로 이동하는 데 사용한다.
  - 프래그먼트는 HTTP 요청 대상에 포함되지 않는다.
- 주소가 달라져도 서버가 그 조건을 사용하지 않으면 응답은 같을 수 있다.

페이지가 표시되는 순서


브라우저
Python 서버
로컬 파일
시간
1 GET /
index.html 읽기
2 200 + HTML
3 HTML 해석
CSS 주소 발견
4 GET /style.css
style.css 읽기
200 + CSS
5 내용 + 스타일
화면 표시
다음 요청 대기

HTTP 요청 읽기


```
GET /search?item=paper HTTP/1.1
Host: 127.0.0.1:8000
Accept: text/html
```

- GET은 자원의 표현을 조회하는 메서드다.
- 요청 경로와 쿼리로 검색 기능과 검색 조건을 전달한다.
- 헤더는 요청에 관한 추가 정보를 담는다.

HTTP 응답 읽기


```
HTTP/1.1 200 OK
Content-Type: text/html; charset=utf-8

<!doctype html>
<html lang="ko"> ... </html>
```

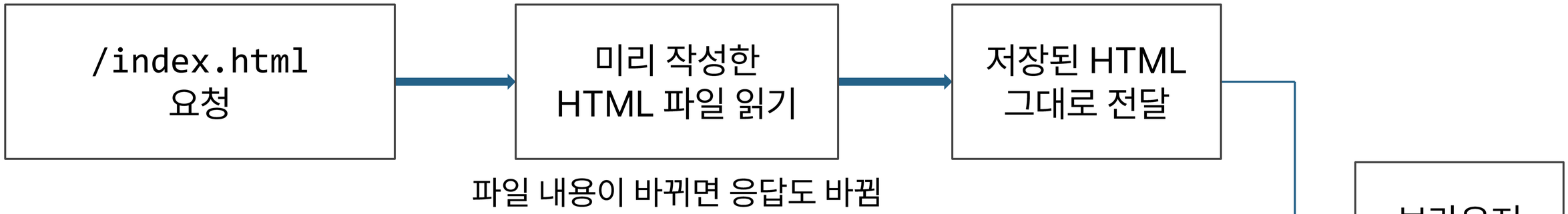
- 상태 코드는 서버가 요청을 어떻게 처리했는지 알려 준다.
- `Content-Type`은 본문의 형식을 설명한다.
- 브라우저는 응답 본문의 HTML을 해석한다.

응답 상태와 연결 실패

상황	관찰 결과	해석
정상 페이지와 정상 검색	200	요청을 성공적으로 처리
유리류 검색 결과가 0건	200	검색 성공, 일치하는 데이터 없음
알 수 없는 품목 값	400	이 실습 서버가 허용하지 않는 입력
`/missing` 경로	404	해당 경로의 자원을 찾지 못함
서버가 실행되지 않음	연결 실패	HTTP 응답 자체를 받지 못함

정적 페이지와 동적 응답

정적 - 파일 작성 / 수정 때 본문을 만듦



파일 내용이 바뀌면 응답도 바뀜

동적 - 요청을 처리할 때 본문을 만듦



입력 / 데이터에 따라 달라질 수 있음



검색 응답을 만드는 과정


```
item = "paper"

results = []
for place in places:
    if place["item"] == item:
        results.append(place)

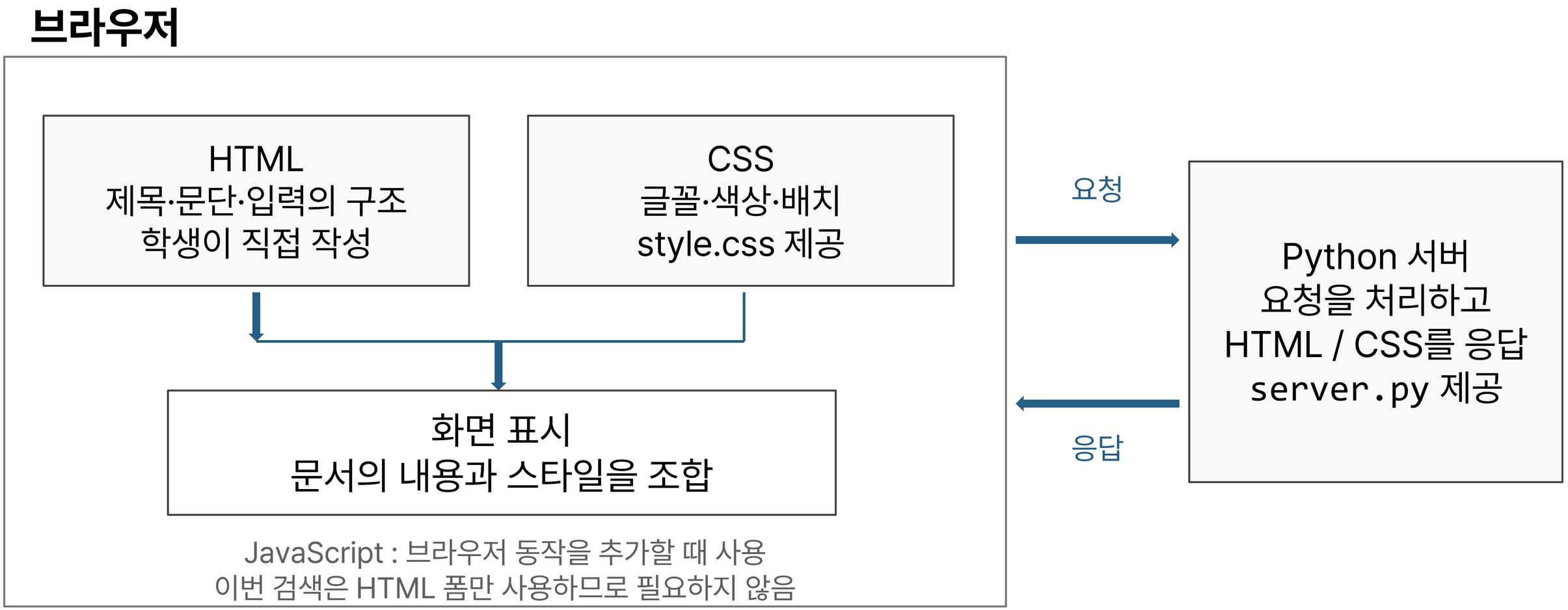
# results를 HTML에 넣어 응답한다.
```

- 요청에서 `item` 값을 읽고 허용한 값인지 확인한다.
- 조건에 맞는 항목을 선택하고 HTML 응답을 만든다.

개발자 도구의 Network

- 개발자 도구를 열고 Network 탭에서 요청 기록을 확인한다.
  - 기록을 지운 뒤 새로고침하고 document 요청을 선택한다.
- Headers에서 Request URL, Request Method, Status Code를 읽는다.
- Response에서 서버가 보낸 HTML을 확인한다.
- 화면의 글자와 요청 조건, 응답 본문을 서로 연결해 설명한다.

HTML, CSS, JavaScript



HTML은 구조, CSS는 표현을 맡고 Python은 서버에서 실행된다.

HTML 문서의 기본 구조


```
<!doctype html>
<html lang="ko">
<head>
  <meta charset="utf-8">
  <title>캠퍼스 분리배출 안내</title>
  <link rel="stylesheet" href="/style.css">
</head>
<body>
  <h1>캠퍼스 분리배출 안내</h1>
  <p>품목에 맞는 장소를 확인합니다.</p>
</body>
</html>
```

요소, 태그, 속성


```
<a href="/search?item=paper">종이류 장소 보기</a>
```

- <a>와 는 시작 태그와 종료 태그다.
- 그 사이의 텍스트가 링크로 표시되는 내용이다.
- href는 이동할 위치를 지정하는 속성이다.
- 요소는 태그와 그 내용을 포함하는 단위다.

중첩과 종료 태그


```
<p>품목은 <strong>종이류</strong>입니다.</p>

<ul>
  <li>A도서관 1층 로비</li>
  <li>B강의동 1층 복도</li>
</ul>
```

- 안쪽에서 연 요소는 바깥 요소보다 먼저 닫는다.
- 들여쓰기는 사람이 구조를 읽기 쉽게 만든다.
- `meta`, `link`, `input` 같은 빈 요소는 종료 태그를 쓰지 않는다.

head와 body의 차이

HTML 문서

<head> 문서 정보

meta charset : UTF-8 문자 해석
title : 캠퍼스 분리배출 안내
link : /style.css 연결

<body> 화면의 내용

h1 : 분리배출 장소 찾기
p : 품목에 맞는 장소를 확인

브라우저에서 확인하는 위치

캠퍼스 분리배출 안내

CSS로 글꼴 / 색상 / 간격 적용

분리배출 장소 찾기

품목에 맞는 장소를 확인합니다.

제목, 문단, 목록


```
<h1>캠퍼스 분리배출 안내</h1>
<h2>해결하려는 문제</h2>
<p>처음 방문한 학생이 분리배출 장소를 찾기 어렵다.</p>
<h2>관찰할 행동</h2>
<ul>
  <li>어디에서 안내를 찾아보는가?</li>
  <li>찾지 못했을 때 어떻게 행동하는가?</li>
</ul>
```

링크와 문서 안의 이동


```
<a href="/search?item=paper">종이류 검색</a>
<a href="#problem">문제 설명으로 이동</a>

<section id="problem">
  <h2>해결하려는 문제</h2>
  <p>관찰한 불편을 적습니다.</p>
</section>
```

의미를 가진 문서 구조


```
<header>
  <h1>캠퍼스 분리배출 안내</h1>
</header>
<main>
  <section><h2>문제 설명</h2> … </section>
  <section><h2>장소 검색</h2> … </section>
</main>
<footer>수업용 가상 데이터입니다.</footer>
```

표로 비교 정보 표현하기

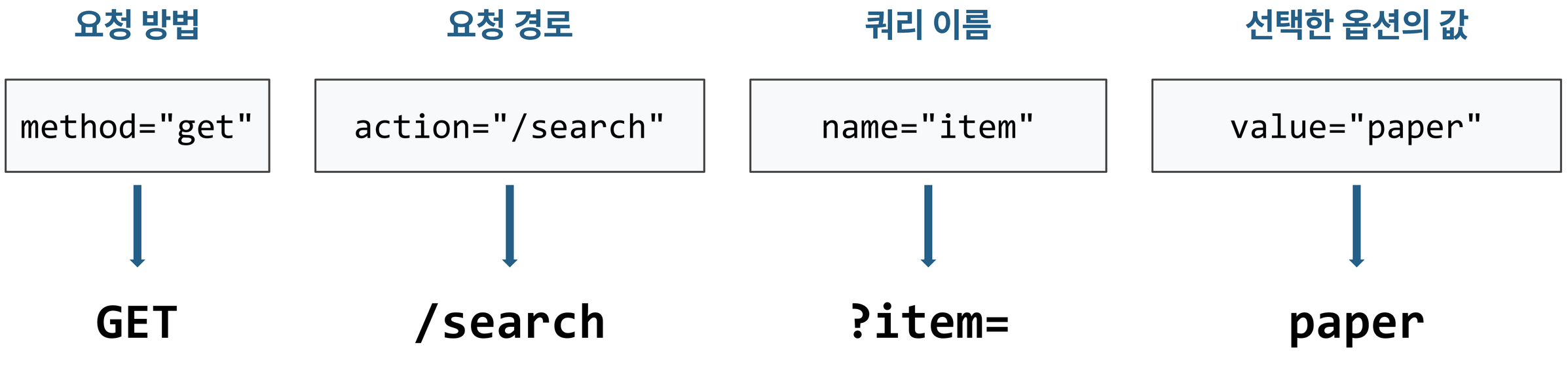

```
<table>
  <tr><th>이해관계자</th><th>확인할 내용</th></tr>
  <tr><td>학생</td><td>안내를 찾는 과정</td></tr>
  <tr><td>시설 담당자</td><td>정보 갱신 가능성</td></tr>
</table>
```

폼으로 검색 조건 보내기


```
<form action="/search" method="get">
  <label for="item">버릴 품목</label>
  <select id="item" name="item">
    <option value="paper">종이류</option>
    <option value="can">캔류</option>
  </select>
  <button type="submit">장소 찾기</button>
</form>
```

폼과 URL의 연결

폼에서 “종이류”를 선택한 경우 : 화면의 글자는 종이류, 전송 값은 paper



생성되는 요청 : `GET /search?item=paper`

검색 폼의 동작 해설

- `action="/search"` + `method="get"` → `GET /search`
- `name="item"` + `value="paper"` → `?item=paper`
- 기본 데이터의 종이류 검색 → 200, A도서관 / B강의동 2건
- `name="material"`로 변경 → 지원하지 않는 이름, 400
- `name` 제거 시 `/search` 요청, 현재 데이터의 전체 결과 응답

개인 실습 목표와 파일

- 각자 `skeleton` 폴더에서 작성 / 실행하고 결과를 확인한다.
- `index.html`의 TODO 1~7 : 제목 / 문단 / 목록 / 표 / 링크 / 검색 폼
- `data.py`의 TODO 8 : Python 리스트에 가상 장소 한 건 추가
- `server.py`와 `style.css`는 제공된 코드를 사용한다.

서버 실행과 첫 접속


```
# Windows
py server.py

# macOS / Linux
python3 server.py
```

- 압축을 해제한 `skeleton` 폴더에서 실행한다.
- 브라우저에서 `http://127.0.0.1:8000/`을 연다.
- HTML은 저장 후 새로고침, Python 데이터는 서버 재시작

실습 1 : 탭과 본문 제목

<title>캠퍼스 분리배출 길잡이</title>

<h1>캠퍼스 분리배출 길잡이</h1>

- TODO 1 : `title`과 `h1`에 서비스 이름을 작성해 보자.
- 저장하고 새로고침한 뒤 탭 제목과 본문 제목을 각각 확인한다.
- 둘 중 하나만 바꿔 보고 표시 위치를 확인한 후 원래 이름으로 맞춘다.

실습 2 : 문제 설명 문단


```
<p>
  이동 수업 중 종이를 버리려는 신입생이
  주변 수거함의 위치와 품목을 알기 어렵다는 가설입니다.
</p>
```

- TODO 2 : 제공된 가상 문제 문장을 p 요소로 작성해 보자.
- 문장을 두 줄로 입력해도 브라우저는 보통 공백을 합쳐 표시한다.
- 한 문단을 더 추가하려면 별도의 p 요소를 사용한다.

실습 3 : 목록과 중첩


```
<ul>
  <li>대상: 이동 수업 중인 신입생</li>
  <li>가상 관찰: 표지를 읽고 다른 층으로 이동함</li>
  <li>추정: 위치 정보가 부족했을 가능성</li>
</ul>
```

- TODO 3 : 제공된 대상 / 가상 관찰 / 추정을 `ul`과 `li`로 작성해 보자.
- `li`는 `ul` 안에 넣고 각 항목의 종료 태그를 확인한다.
- 새 `li` 하나를 추가했다가 삭제하며 목록의 변화를 확인한다.

실습 4 : 표의 행과 셀


```
<tr>
  <td>신입생</td>
  <td>사용자 후보: 이동 중 필요한 위치 확인</td>
</tr>
```

- TODO 4 : 제공된 표 머리글 아래에 이해관계자 3행을 작성해 보자.
- 사용자, 운영자 후보, 정보 갱신 담당 후보를 각각 한 행에 넣는다.
- `tr` 안에 `td` 두 개를 넣고 브라우저에서 열의 대응을 확인한다.

실습 5 : 문서 안의 링크


```
<a href="#sample-note">가상 데이터 안내 보기</a>

<section id="sample-note">
  <h2>가상 데이터 안내</h2>
  ...
</section>
```

- TODO 5 : 안내 영역의 id와 a의 href를 연결해 보자.
- 링크를 누르면 주소에 `#sample-note`가 붙고 해당 영역으로 이동한다.
- 연결이 확인되면 링크 문구를 "가상 데이터 안내 보기"로 정한다.

실습 6 : GET 검색 폼


```
<form action="/search" method="get">
  <label for="item">찾을 품목</label>
  <select id="item" name="item">
    <option value="paper">종이</option>
  </select>
  <button type="submit">장소 찾기</button>
</form>
```

- TODO 6 : `label`, `select`, `option`, `button`을 `form` 안에 작성해 보자.
- `option` 값은 `all`, `paper`, `plastic`, `can`, `glass`로 넣는다.
- 종이를 선택하면 `/search?item=paper`로 이동하고 2건이 표시된다.

실습 7 : 빠른 검색 링크


```
<a href="/search?item=paper">종이 장소 바로 보기</a>
<a href="/search?item=glass">유리 장소 바로 보기</a>
```

품목	쿼리	시작 데이터 결과
전체	`?item=all`	5건
종이	`?item=paper`	2건
플라스틱	`?item=plastic`	2건
캔	`?item=can`	1건
유리	`?item=glass`	0건

실습 8 : Network 확인

- 개발자 도구에서 Network를 열고 기록을 켠다.
- 처음 화면을 새로고침하고 문서 요청과 `/style.css`를 구분한다.
- 종이 검색 요청의 `URL`, `GET`, `200`, `Content-Type`을 확인한다.
- Response에서 결과 HTML과 A도서관 / B강의동을 찾는다.
- 브라우저 화면, Network 응답 본문, 서버 터미널 로그를 대조한다.

실습 9 : 응답 상태 비교

직접 요청할 경로	상태	응답에서 확인할 내용
`/search?item=paper`	200	종이 장소 2건
`/search?item=glass`	200	등록된 결과 0건
`/search?item=unknown`	400	지원하지 않는 검색 조건
`/missing`	404	처리할 경로 없음

실습 10 : 폼 오류 수정

- `name="item"`을 `name="material"`로 바꾸면 어떻게 되는가?
- `name`을 복원하고 `action`을 `/missing`으로 바꾸면 어떻게 되는가?
- `action`을 복원하고 종이 검색 2건을 다시 확인한다.
- `name`을 완전히 지우면 이 폼은 쿼리 없이 요청하여 전체 결과를 받는다.
- 마지막에는 원래 상태로 모두 복원한다.

실습 11 : 데이터 수정

{"item": "can",
"place": "D강의동 1층 로비",
"note": "캔 수거함이 있다고 가정합니다."},

- TODO 8 : `data.py`의 `PLACES` 끝에 가상 장소 한 건을 추가해 보자.
- 저장 후 Ctrl+C로 서버를 종료하고 같은 명령으로 다시 실행한다.
- 전체는 5건에서 6건, 캔은 1건에서 2건으로 바뀌는지 확인한다.

감사합니다.

utilForever@gmail.com

https://github.com/utilForever

Facebook, Twitter, Instagram, LinkedIn: utilForever